Metodología de la Programación

Grado en Ingeniería Informática

Curso 2025/2026

Romero Oliva, Christian

Muñoz Arillo, Javier

Pérez Tejero, José Miguel

Proyecto Modularidad

ESI-ESCAPE

Índice general

Índice general **1**
1. Introducción 2
2. Documentación de usuario 4
   2.1. Descripción funcional 4
   2.2. Tecnología 4
   2.3. Manual de instalación 4
   2.4. Acceso al sistema 5
   2.5. Manual de referencia 6
   2.6. Guía del operador 7
3. Documentación de sistema 8
   3.1. Especificación del sistema 8
   3.2. Módulos 8
   3.3. Plan de prueba 9
4. Documentación del código fuente 17

1. Introducción

Este es un ejemplo de plantilla para la generación de la documentación asociada al desarrollo de un programa. Se puede usar como guía, pero puede sufrir todas las modificaciones que considere oportunas para mejorar su documento.

Es necesario incluir una portada, dispone de cuatro ejemplos. Puede comprobar el resultado de cada uno de ellos en los ficheros PDF. En el fichero .tex está por defecto el ejemplo de Portada1.

Se debe añadir un índice. Puede observar en el código cómo se puede incluir en la sección destinada a tal fin.

A lo largo del documento también puede surgir la necesidad de añadir una imagen. Puede ver cómo hacerlo en el código de las portadas.

LaTeXtambién le permite incluir código. Fíjese en el código fuente cómo está delimitado. El resultado para el típico ejemplo *Hola mundo* en C es el siguiente:

```
/* Programa: Hola mundo */

#include <stdio.h>

int main()
{
    printf( "Hola mundo." );
    system("PAUSE");
    return 0;
}
```

Puede consultar tutoriales, libros o blogs para profundizar más en el aprendizaje de LaTeX, resulta muy interesante por la gran ventaja que ofrece de proporcionar documentos con una alta calidad. Sea cual el sistema operativo tenga en cuenta que va a neccesitar:

1. Editor de texto para generar el fichero .tex con el código
2. Distribución de LaTeXpara compilar el fichero .tex y generar el fichero .pdf
3. Visor de documentos para poder visualizar el fichero .pdf generado

Las siguientes secciones indican cada una de las partes que se debe incluir en la documentación del programa. Hay que tener en cuenta que se trata de una tarea tediosa pero muy necesaria para realizar un buen mantenimiento

del programa. Para cualquier modificación del código hay que entender el mismo, siendo de gran ayuda la documentación.

Recuerde que en la documentación del programa se distinguen dos partes claramente diferenciadas:

- Documentación interna. Documentación contenida en las líneas de comentarios del programa.
- Documentación externa. Documentación que incluye el análisis, diagramas de flujos y/o pseudocódigos, manuales de usuario con instrucciones para ejecutar el programa e interpretar los resultados. Dentro de esta se distingue a su vez:
    - Documentación del usuario
    - Documentación del sistema

En las siguientes secciones se describirá el contenido a incluir en cada uno de estos tipos de documentación. El objetivo que se debe marcar en la redacción de las mismas debe ser el de conseguir un texto claro, completo y conciso, utilizando para ello construcciones gramaticales y ortográficas correctas.

2. Documentación de usuario

Documentación relacionada con las funciones del sistema pero no se hace referencia a nada relativo a su implementación. Está orientado a las personas que van a usar el sistema no a los que lo van a mantener.

2.1. Descripción funcional

- Debe describir de forma simple el propósito del sistema.
- En esta sección se debe incluir una breve descripción funcional sobre lo que hace y no hace el sistema.
- Se pueden incluir pequeños ejemplos para dar una visión general de las características del sistema.

2.2. Tecnología

Las tecnologías y herramientas usadas para el desarrollo de este proyecto de juego de escape-room son:

- Git: Usado para la administración de los archivos del proyecto, control de versión, y compartido de archivos para trabajo en equipo.
- Visual Studio Code: Editor de código con integración de Git para ver los cambios realizados y resolver conflictos donde varios miembros del equipo suban modificaciones que colisionen una con la otra.
- Librerías: Se usan las librerías del estándar ANSI C, las cuales son empleadas para constituir módulos que realizan las distintas tareas que requiere el proyecto.

2.3. Manual de instalación

Para la instalación de nuestra implementación de ESI-Escape, el usuario deberá ejecutar desde la terminal **(¡MUY IMPORTANTE, hacerlo estando dentro del directorio del proyecto, para asegurar la carga de archivos con las rutas correctas!)** "run" (para directamente ejecutar el proyecto si está ya compilado en la carpeta "bin"), "run_and_compile" (para compilar el proyecto y luego ejecutarlo) o "compile" (para solo compilarlo en la carpeta "bin"). Se usarán los archivos .bat si el usuario se encuentra en el sistema operativo Windows y los archivos .sh si se encuentra en Linux.

Para la compilación, es necesario que el usuario tenga instalado "make" mediante la forma preferida y disponible del usuario (en Linux lo instalamos usando el administrador de paquetes preferido para instalar "build_essential", pero cabe notar que la instalación de esta herramienta varía dependiendo del tipo de distribución, y muchas veces viene preinstalada), y en Windows que tenga instalado "MinGW" con su carpeta "bin" añadida al PATH para la llamada que hacen "run_and_compile" y "compile" de "mingw32-make", el cual cumple la misma función que "make": compilar todo el proyecto con sus archivos de cabecera .h y de código .c.

(Los archivos .txt de datos de entrada y de guardado de partida y jugadores se encuentran en el directorio "data").

2.4. Acceso al sistema

Al entrar al programa, se le pedirá al usuario introducir su nombre de acceso y contraseña. Si estos no son encontrados, se le ofrece el usuario registrarse, pidiendo adicionalmente un nombre completo para este. Una vez iniciado sesión, el usuario verá el menú de inicio de ESI-ESCAPE, con las opciones de comenzar una Nueva partida, Cargar partida y Salir (para salir del juego de vuelta al sistema operativo). Durante una partida, la opción "Volver"te permitirá salir al menú de inicio para entonces dejar el juego.

El usuario se puede encontrar con el problema de encontrarse mensajes de alerta con archivos los cuales no pudieron ser cargados. En esos casos, la solución radica en ejecutar "run" o "run_and_compile" situándonos con la terminal en la carpeta del proyecto (es decir, la misma que contiene dichos archivos), para que así ESI-Escape encuentre los archivos .txt.

También puede ocurrir que el sistema operativo no encuentre "bin/esi-escape" o "mingw32-make" o "make".

1. Si no se encuentra "bin/esi-escape" al ejecutar "run", significa que no está compilado, o no se está ejecutando "run" o "run_and_compile'desde la carpeta del juego'. Se soluciona ejecutando "run", "run_and_compile" o "compile" desde la carpeta del juego, y asegurándonos de que el ejecutable está presente en "bin" haciendo "run_and_compile" o "compile".

2. Si no se encuentra "mingw32-make", instalar MinGW en tu equipo Windows, o si estás usando Linux, es recomendable instalar "make" en su lugar mediante la forma preferida y disponible del usuario (en Linux lo instalamos usando el administrador de paquetes preferido para instalar "build_esential").

3. Si no se encuentra "make", y estando en Linux, lo instalamos mediante la forma preferida y disponible del usuario (en Linux lo instalamos usando el administrador de paquetes preferido para instalar "build_essential", pero cabe notar que la instalación de esta herramienta varía dependiendo del tipo de distribución, y muchas veces viene preinstalada).

2.5. Manual de referencia

La interfaz de juego de ESI-Escape permite una navegación fluida entre las salas y sus objetos y puzles; permitiendo realizar diversas acciones sobre estos.

- Las opciones de "Describir sala", "Examinar (objetos y salidas)" e "Inventario" mostrarán información sobre lo que sus nombres indican, para luego pedir al usuario una pulsación para volver al menú de juego.
- "Entrar en otra sala" te permitirá ir a otra sala la cual no esté bloqueada, esté adyacente a la actual y exista.

  "Coger objeto" te permitirá agarrar un objeto que exista y esté presente en la sala actual.

  "Soltar objeto" permite colocar un objeto que exista y esté en tu inventario en la sala actual.

  "Usar objeto" permite usar un objeto que exista y esté en tu inventario para desbloquear salidas que requieran de dicho objeto para desbloquearse, estén adyacentes a la actual y existan.

  Y "Resolver puzle / introducir código" permite al usuario resolver un puzle o introducir un código que exista y se encuentre en la sala actual, para así desbloquear las salidas que lo tengan como condición en cualquier parte.

  Si cualquiera de las condiciones especificadas anteriormente no se cumplen, el juego mostrará mensajes de error haciendo saber al jugador lo que está incumpliendo.

- Al pulsar "Guardar partida", los datos de la partida actual son almacenados en el directorio "data" para que el usuario pueda cargar su partida en una próxima sesión.
- Por último, si pulsas "Volver", el usuario volverá a la pantalla de inicio donde podrá crear una Nueva partida, Cargar partida o Salir del juego.

2.6. Guía del operador

(Este programa no requiere de operador)

3. Documentación de sistema

Documentación referida a los aspectos del análisis, diseño, implementación y prueba del *software*, así como la implantación del mismo. En general, debe hacer referencia al sistema, ya que está orientada a los programadores que van a realizar el mantenimiento. Debe estar muy bien estructurada, desde lo más general a los más interno.

3.1. Especificación del sistema

Esta sección debe describir el análisis y la especificación de requisitos. Cómo se descompone el problema en distintos subproblemas y los módulos asociados a cada uno de ellos, acompañados de su especificación. También debe incluir el plan de desarrollo del *software*.

3.2. Módulos

A continuación se presenta una breve descripción de cada uno de los módulos en las que ESI-Escape está divido.

- **Partida**: Es el encargado de gestionar tareas relacionadas con la partida en sí. Esto es, guardar y cargar su estado, inicializar los archivos de configuración en memoria mediante la importación de las funciones de carga de cada módulo, y gestionar el estado de la estructura "GameState", que almacena el estado de la partida.

- **Inventario**: La responsabilidad de este módulo es la gestión de los objetos. Interacciona con el archivo de configuración **Objetos.txt** para cargar los objetos registrados, y alberga diferentes funciones para buscar objetos en memoria, actualizar su estado y escribirlos al guardar la partida.

- **Salas**: Este módulo se encarga del administrado tanto de las conexiones como de las salas que componen el escape room. Posee funciones para la carga de conexiones y salas; liberarlas de la memoria, obtener una sala por su ID, y obtener la sala inicial de la partida actual. Este módulo muestra en pantalla errores si no se pudieron cargar los datos de las salas y/o conexiones, o si poseen un formato incorrecto en alguno de sus campos.

- **UI (Interfaz de Usuario)**: Este módulo se centra en pedir información al usuario y mostrarla por pantalla. Posee funciones para la

creación de menús, vaciar el buffer de entrada, esperar a que el usuario presione una tecla para continuar, elegir SÍ o NO, mostrar el logo del juego y otros elementos por pantalla; salir del juego, mostrar los menús de registro o inicio de sesión, y el menú principal (el módulo '**partida** usa sus funciones para crear el menú de juego); mostrar los datos de las estructuras cargadas por memoria dinámica con la opción de poner filtros (por ejemplo, para solo ver los objetos en la sala actual); mostrar las interfaces de las diferentes opciones del menú de juego y de salir del juego mostrando unos breves créditos con los miembros creadores del trabajo.

3.3. Plan de prueba

Debe describir todo el plan de pueba que se ha llevado a cabo. Se distinguen las pruebas realizadas a cada módulo y las pruebas de integración de los distintos módulos probándolo como un todo. Por último, también se debe incluir la descripción del plan de pruebas de aceptación.

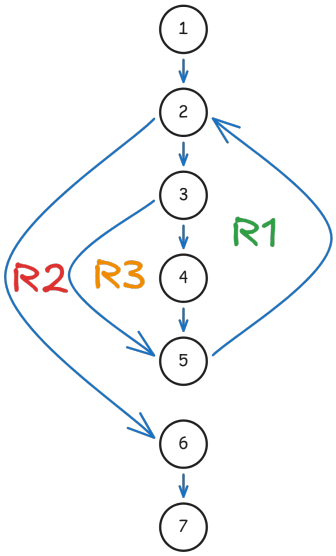
Pruebas de ruta básica (caja blanca) de tres funciones

- **Pruebas de ruta básica (caja blanca) de "salas_get_sala_from_id" (José Miguel Pérez Tejero):**

Código:

```
Sala* salas_get_sala_from_id(char* id_to_search, Salas* arr_salas){

    int i;
    Sala* Sala_return = NULL; (1)

    for(i=0; i<(arr_salas->number_of_salas); i++){ (2)

        if(strcmp(arr_salas->salas[i].sala_id, id_to_search) == 0){ (3)

            Sala_return = &(arr_salas->salas[i]); (4)

        } (5)

    } (6)

    return(Sala_return); (7)

}
```

Diagrama de control de flujo:



Complejidad ciclomática:

V(G) = número de regiones (R1, R2 y R3) = 3

Casos de prueba:

- Ruta 1 (1-2-6-7) (No entramos en el for):

  Para llegar a un caso en el que no entremos al for, el número asociado al número de salas debe ser menor que 1.
  Así pues, un caso de prueba sería en el cual "`number_of_salas`" es menor que 1.
  El resultado esperado es que la función devuelva "`NULL`".

- Ruta 2 (1-2-3-5-2-6-7) (Entramos en el for pero no entramos en el if):

  Para entrar en esa ruta, no debe de haber ninguna sala que comparta el valor de "`id_to_search`", y "`number_of_salas`" debe ser mayor o igual que 1.
  También probaremos casos donde "`number_of_salas`" supere al número real de salas.
  Para estos casos de prueba el resultado esperado será que la función devuelva '`NULL`'(o, en el caso de "`number_of_salas`" mayor que el número real de salas, que al buscar la ID en un espacio de memoria en el que no haya realmente salas, justo coincida que la memoria basura contenga el valor de la ID buscada, en cuyo caso se obtendrá una sala inexistente, ocasionando que el programa use memoria basura como datos de juego, y que al usarse esta función, el juego deje de comportarse de manera adecuada, mostrando texto incoherente al describir sala, por ejemplo).

- Ruta 3 (1-2-3-4-5-2-6-7) (Entramos en el for y en el if):

  Aquí "`number_of_salas`" será mayor o igual que 1, coincidirá con el número de salas, y habrá una sala mínima cuyo ID coincida con "`id_to_search`".
  Así pues, diseñaremos casos de prueba que cumplan dichas condiciones, y dichos casos deberán hacer que la función devuelva un puntero a la sala con la ID buscada.

Pruebas de caja negra de las funciones de cada módulo y de cada módulo como conjunto:

- **Pruebas del módulo UI y sus funciones:**

  - `void ui_clean_buffer();` : Probaremos el caso en el que queden en el buffer caracteres seguidos de un salto de linea `\n` (Se esperará el resultado de que el buffer se vacíe), y otro caso a probar será en el cual el buffer contenga la señal `EOF` (en cuyo caso se espera que se termine de vaciar el buffer sin problema).

- `void ui_graphic_show_game_name();` : Simplemente probaremos el caso en el que llamaremos a la función, siendo el resultado esperado que se muestre por pantalla el nombre del juego.
- `void ui_graphic_show_screen_separation();` : Simplemente probaremos el caso en el que llamaremos a la función, siendo el resultado esperado que se muestre por pantalla una separación en forma de línea.
- `int ui_menu_create(Menu);` : Probaremos a crear un menú y elegir una opción con un número dentro del rango de posibilidades (se espera que se devuelva el índice adecuado y/o se ejecute la acción asignada a la opción si no es nula) y también probaremos a, con el mismo menú, introducir una letra, caracter o un número fuera de rango (el resultado esperado será que se vuelva a solicitar la entrada al usuario "`Elige dentro del rango de opciones >`").
- `int ui_confirmation(char text[200]);` : Probaremos a introducir s o S (la función deberá devolver un 1), también a introducir n o N (la función deberá devolver 0) y por último, introducir cualquier otro caracter (siendo el resultado esperado una nueva solicitud de entrada al usuario "`(S/N) >`").
- `void ui_anykey_press();` : Simplemente probaremos a llamar a la función, y esperaremos a que el usuario pulse una tecla (el resultado esperado será que el flujo del programa continue sin problemas tras la pulsación).
- `void ui_exit_game(GameState*, int);` : Probaremos el caso en el que salgamos del juego son el parámetro entero "`save_to_jugadores`" a 1 (siendo el resultado esperado el guardado de los datos del jugador junto con sus objetos en Jugadores.txt, el liberado de memoria dinámica, el mostrado de los créditos, y una salida con código 0, de éxito), y también el caso en el que "`save_to_jugadores`" sea 0 (en cuyo caso simplemente se liberará la memoria dinámica, aparecerán los créditos, y se saldrá del juego con código 0, de éxito).
- `void ui_user_initial_menu(GameState*);` : Los casos de prueba que podemos comprobar son los de ingresar un usuario y contraseña existentes en Jugadores.txt (siendo el resultado esperado que el jugador correspondiente sea asignado como jugador actual y se imprima la bienvenida), que al no existir las credenciales se pida y se acepte registrar un nuevo jugador (en cuyo caso se pedirá el nombre completo y se guardará en Jugadores.txt) y por último

el caso en el que, en el que en vez de aceptar, se rechace el registro (ocurriendo que salimos del juego).

- `int ui_main_menu();` : Probaremos cada una de las opciones de Nueva partida, Cargar partida y Salir (se deberá de comenzar un nuevo juego con el jugador actual, cargar la partida guardada del jugador actual y empezar a jugar, o simplemente salir del juego respectivamente).
- `void ui_describe_sala(Sala*, GameState*);` : Probaremos el caso en el que sala sea de tipo INICIAL (se mostrarán el nombre, descripción y el hecho de que es una sala inicial), que sea una sala NORMAL (se mostrarán el nombre, descripción y el hecho de que es una sala normal) y que sea una sala de SALIDA (se imprimirá un texto de VICTORIA y se preguntará si se quiere volver al menú principal).
- `void ui_show_filter_connections(Conns*, Salas*, char*);` : Probaremos los casos de prueba en los cuales pasaremos una sala con conexiones (se mostrará el destino y el estado de bloqueo de cada una), y una sin conexiones (no se imprimirá ninguna conexión).
- `void ui_show_filter_inventory(Inventory*, char*);` : Se probará introducir un filtro existente de localización con objetos, o una cadena vacía "" (se mostrarán los objetos de esa localización o todos respectivamente) y a poner un filtro donde no hayan objetos (se mostrará - VACIO -).
- `void ui_examine_sala(Sala*, GameState*);` : Simplemente probaremos el caso en el que llamaremos a la función (siendo el resultado esperado que se muestren los objetos y las conexiones de la sala, esperando una pulsación de teclado del usuario al final).
- `void ui_enter_sala(GameState*);` : Probaremos a introducir un ID de destino accesible (en cuyo caso avanzaremos de sala), una ruta bloqueada (mostrando que ésta está bloqueada, y hay una condición para desbloquearla), introducir un destino inexistente (mostrandose un mensaje de error "NO HAY NINGUNA SALIDA A ESA SALA, O NO EXISTE!"), e introducir `n` o pulsar ENTER (volviendo así atrás sin hacer nada).
- `void ui_pick_drop_object(GameState*, int);` : Probaremos con el parámetro entero "`pick`" a 1 (pasando así un objeto de la sala al inventario pidiendo confirmación), con su valor a 0 (pasando

un objeto del inventario a la sala actual, también pidiendo confirmación), o introduciendo un ID que no esté disponible (mostrando "ESTE OBJETO NO ESTÁ EN ESTA SALA!" o "NO TIENES ESTE OBJETO!" dependiendo de la ocasión).

- `void ui_show_player_inventory(GameState*);` : Probaremos simplemente a llamar esta función (ocurriendo que se muestren los objetos con localización "Inventario").
- `void ui_use_object(GameState*);` : Probaremos la función estando en una sala que tenga una salida desbloqueable (ocurriendo que el "`conn_block`" de la conexión que requería de ese objeto pase a 0 y se nos notifique de ello), y también usar un objeto que no se pueda usar en esa sala (imprimiendose que ese objeto no se puede utilizar en ese lugar), además de probar por supuesto con objetos que se tengan, no se tengan o que no estén en la sala actual (notificándose los últimos dos casos como errores).
- `void ui_solve_puzzle(GameState*);` : Podemos probar a introducir una ID de puzle que no exista (notificándose como error), y luego, introduciendo puzles que existan, podemos resolverlo correctamente (desbloqueando todas las salidas relacionadas a este y marcando `resuelto=1`) a resolverlo incorrectamente (se notificará la incorrectitud y no se alterarán las conexiones) y que el puzle esté resuelto ya de antes pues `resuelto=1` (ocurriendo que se comunique que ya está resuelto dicho puzle).

- **MÓDULO COMPLETO**: Podemos probar una secuencia en la que ejecutemos `ui_user_initial_menu` y `ui_main_menu` (cargandose o registrandose el jugador especificado y realizando la opción de nueva o cargar partida, o bien saliendo del programa). También podremos probar funciones como `ui_examine_sala`, `ui_pick_drop_object`, `ui_show_player_inventory`, `ui_use_object`, `ui_solve_puzzle`, o `ui_enter_sala` (viendo que las funciones del menú enmedio de una partida funcionan como es debido y realizan sus funciones). Y por último, podremos probar a ejecutar `ui_exit_game` (para ver que funciona correctamente el guardado de datos de jugadores, el liberado de memoria, el mostrado de créditos, y el cerrado del juego).

- **Pruebas del módulo salas y sus funciones:**
  - `Sala* salas_get_sala_from_id(char*, Salas*);` : Podemos probar a buscar una sala con ID existente (ocurriendo que se devuelva

un puntero a la sala con el ID buscado), y a buscar por un ID que no exista en ninguna sala (devolviendo así `NULL`).

- `Sala* salas_get_sala_inicial(Salas*);` : Probaremos con un conjunto de salas con sala inicial (devolviendo así un puntero a dicha sala) y también un conjunto sin sala inicial (devolviéndose `NULL`):
- `Salas salas_load_salas(char*);` : Podemos probar con un archivo con una ruta que exista y con formato correcto (cargándose sin problemas todos los datos de las salas), con un archivo inexistente o sin permisos (mostrando un mensaje de error y saliendo de del juego con código 1, de error), o también con unas salas con tipos de salas incorrectos o mal formateados (notificándose que hay un formato incorrecto de tipo de sala y saliendo de del juego con código 1, de error).
- `Conns salas_load_conns(char*);` : Los casos de prueba serían probar con un archivo con una ruta que exista y con formato correcto (cargándose sin problemas todos los datos de las conexiones), con un archivo inexistente o sin permisos (mostrando un mensaje de error y saliendo de del juego con código 1, de error), o también con unas conexiones con formateado de bloqueo o activación incorrectos (notificándose "Formato incorrecto de bloqueo o activacion de conexion." y saliendo de del juego con código 1, de error).
- `Conn* salas_find_conn_by_id(char*, Conns*);` : Probaremos a buscar una conexión con un ID que exista (ocurriendo que se devuelva un puntero a la conexión con el ID buscado), y a buscar por un ID que no exista en ninguna conexión (devolviendo así `NULL`).
- `void salas_free_salas(Salas*);` : Podemos encontrar los casos de prueba de que tengamos un array de Salas ya cargado (liberándose sin problema) y también pasar un array de Salas cuyo puntero esté con valor NULL (lo cual no ocasionará ningún fallo ni cierre accidental del programa).
- `void salas_free_conns(Conns*);` : Probaremos los casos de prueba en los que tengamos un array de conexiones ya cargado en memoria (liberándose sin problema) y también pasar un array de conexiones cuyo puntero esté con valor NULL (lo cual no ocasionará ningún fallo ni cierre accidental del programa).

- **MÓDULO COMPLETO**: Podemos probar a primero ejecutar `salas_load_salas` y `salas_load_conns`, luego `salas_get_sala_inicial` para obtener el punto de inicio, buscar en las conexiones una que tenga como punto de partida la sala inicial y ver el destino, para usar `salas_get_sala_from_id` y obtener el id de dicho destino, y terminar liberando memoria con `salas_free_salas` y `salas_free_cons` (debiendo de ocurrir todo el proceso sin ningún fallo ni ningún conflicto de integración entre cada función). También podemos intentar usar las funciones `salas_load_salas` y `salas_load_conns` con datos incorrectamente formateados o archivos no existentes o sin permisos (parando el programa porque, o bien no se puedo abrir el archivo pedido, o no estaba bien formateado).

Prueba de integración

Incluir toda la batería de pruebas realizadas. Pruebas de caja blanca y pruebas de caja negra.

Plan de pruebas de aceptación

Estas pruebas deben diseñarse con el usuario del sistema. Deben describir las pruebas que son necesarias pasar para que el sistema sea aceptado por el usuario final.

Si en algún punto del documento se quiere hacer referencia a algún documento es necesario incluir la cita donde corresponda, podemos tomar como ejemplo **?**.

4. Documentación del código fuente

En este punto se puede incluir toda la documentación interna que puede ser generada con Doxygen.

La documentación interna del programa, como se ha comentado anteriormente, comprende los comentarios del programa. Durante la fase de implementación es necesario comentar adecuadamente cada una de las partes del programa. Estos comentarios se incluyen en el código fuente con el objetivo de aclarar qué es lo que hace el fragmento de código, para explicar sus elementos.

Esta documentación ayuda a hacer más comprensible el código fuente a los programadores que deban trabajar en él, facilitando las tareas de mantenimiento. No sólo proporciona una mayor legibilidad en la actualización y modificación del código, sino también en la depuración del mismo.